Tu es un dévelop

Aujourd'hui 00:16 projet web

Tu es un développeur senior MERN Stack (MongoDB, Express, React, Node.js).
Ta mission est de concevoir et implémenter un système de vente de services digitaux
(structuré, scalable et maintenable), destiné à une plateforme de services IMEI
et d'activation de logiciels.

OBJECTIF GÉNÉRAL

Concevoir un système unifié de services et de commandes, capable de gérer plusieurs
types de services avec des comportements différents, tout en conservant une seule
logique backend et des formulaires dynamiques côté frontend.

═══════════

TYPES DE SERVICES (CATÉGORIES SYSTÈME) - DÉFINITION FIXE
Le système doit gérer exactement 4 catégories de services fixes :

1) IMEI SERVICE
  - Champs utilisateur requis:
    * IMEI (15 chiffres)
    * URL image (lien imgbb.com - utilisateur colle l'URL)
  - Traitement admin: Manuel
  - Livraison: Changement de statut à "completed" (aucune donnée supplémentaire)
  - Notification: Email de

confirmation

2) SERVER SERVICE
- Champs utilisateur requis:
  * Username (logiciel)
  * Email (compte logiciel)
- Traitement admin: Activation manuelle du compte
- Livraison: Changement de statut à "completed" (aucune donnée supplémentaire)
- Notification: Email de confirmation

3) RENTAL SERVICE
- Champs utilisateur: AUCUN
- Traitement admin: Génération/

saisie des identifiants
- Livraison:
  * Admin saisit: username et password
  * Données visibles dans l'espace utilisateur
  * Email envoyé avec credentials
- Notification: Email avec identifiants

4) LICENSE SERVICE
- Champs utilisateur: AUCUN
- Traitement admin: Génération/saisie de la clé d'activation
- Livraison:
  * Admin saisit: activationKey
  * Clé visible dans l'espace

utilisateur
    * Email envoyé avec clé
  - Notification: Email avec clé d'activation

Ces catégories sont définies au niveau système et déterminent :
- le formulaire affiché côté utilisateur
- le workflow de traitement côté admin
- le type de livraison finale

═══════════════════════════════════════
═══════════════════════════════════════
═══════════════════════════════════════
═══════

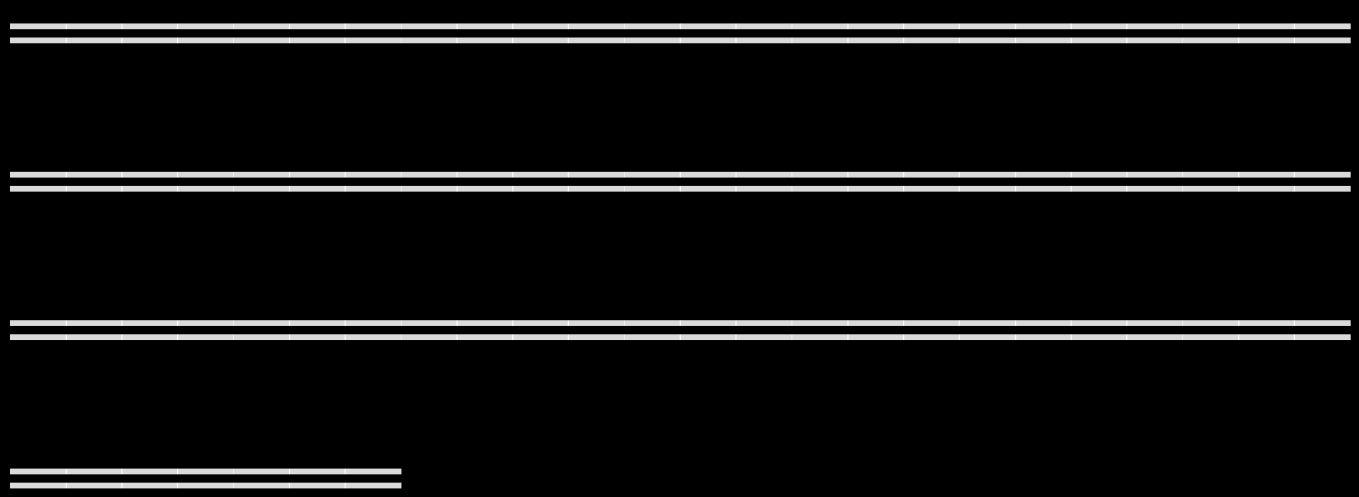
GESTION DES SERVICES (CÔTÉ ADMIN)

L'administrateur doit pouvoir :

- créer, modifier et désactiver des services
- renseigner le nom, la description et le prix du service
- sélectionner une catégorie de service via une liste prédéfinie (IMEI, Server, Rental, License)

L'administrateur ne définit jamais de formulaires manuellement.

La catégorie sélectionnée pilote automatiquement le comportement du service.



AFFICHAGE DES SERVICES (CÔTÉ UTILISATEUR)

Les services doivent être affichés par pages dédiées, une par catégorie :

- /services/imei
- /services/server
- /services/rental
- /services/license

Chaque page affiche uniquement les

services correspondant à sa catégorie.
Ces pages sont des filtres visuels basés sur une logique backend unique.

═══════════════════════════════════════

SYSTÈME DE COMMANDE (UNIFIÉ)
Le système doit utiliser une seule entité/collection "Order".
Aucune séparation par type de commande n'est autorisée.

Chaque commande doit contenir :
- l'utilisateur (userId)
- le service commandé (serviceId)
- la catégorie du service (copiée lors de la commande)
- le statut de la commande (pending, processing, completed)
- les données saisies par l'utilisateur si applicables (userSubmittedData)
- les données de résultat/livraison si applicables (deliveryData)
- la date de création (createdAt)

═══════════

FORMULAIRE DE COMMANDE (DYNAMIQUE)
Il doit exister une seule page de commande.
Le formulaire affiché dépend uniquement de la catégorie du service :

- IMEI : champs IMEI (15 chiffres) + URL image (lien imgbb.com)
- Server : champs username + email
- Rental : aucun champ (direct checkout)
- License : aucun champ (direct checkout)

Le formulaire ne doit jamais être codé en dur.
Il doit être généré dynamiquement selon la catégorie du service.

═══════════════════════════════════════
═══════════════════════════════════════
═══════════════════════════════════════
═══════════

SYSTÈME DE PAIEMENT
Une fois la commande passée :
- Vérifier que l'utilisateur a un solde suffisant (solde ≥ prix du service)
- Si oui: créer la commande et

débiter le montant du solde utilisateur immédiatement
- Si non: afficher erreur et empêcher la création de la commande
- Retourner le nouveau solde au frontend après débit

═══════════════════════════════════════
═══════════════════════════════════════
═══════════════════════════════════════
═══════

HISTORIQUE DES COMMANDES (ADMIN ET UTILISATEUR)

CÔTÉ ADMIN:

- Pages d'historique par catégorie :
  /admin/orders/imei
  /admin/orders/server
  /admin/orders/rental
  /admin/orders/license
- Chaque page filtre les commandes selon la catégorie
- L'admin peut:
  * Consulter les données soumises par l'utilisateur
  * Traiter la commande
  * Saisir les données de livraison (pour Rental et License uniquement)
  * Changer le statut (pending → processing → completed)
- Pour Rental/License: Modal ou formulaire pour saisir credentials/clé

- Pour IMEI/Server: Simple bouton "Marquer comme complétée"

CÔTÉ UTILISATEUR:
- Pages d'historique par catégorie :
  /account/orders/imei
  /account/orders/server
  /account/orders/rental
  /account/orders/license
- L'utilisateur peut:
  * Consulter le statut de sa commande (pending, processing, completed)
  * Pour Rental/License: Voir les identifiants/clé une fois la commande complétée
  * Pour IMEI/Server: Voir seulement

le statut "completed"

═══════════════════════════════════════

═══════════════════════════════════════

═══════════════════════════════════════

═══════

WORKFLOW DE TRAITEMENT DÉTAILLÉ

IMEI SERVICE & SERVER SERVICE:

1. Utilisateur saisit données + passe commande
2. Commande créée avec statut "pending"
3. Solde débité immédiatement

4. Admin voit commande dans /admin/orders/[category]
5. Admin traite manuellement (déblocage ou activation)
6. Admin clique "Marquer comme complétée"
7. Statut devient "completed"
8. Email notification envoyée à l'utilisateur
9. Utilisateur voit "completed" dans /account/orders/[category]

RENTAL SERVICE & LICENSE SERVICE:

1. Utilisateur clique "Passer commande" (aucun formulaire)
2. Commande créée avec statut

"pending"

3. Solde débité immédiatement

4. Admin voit commande dans /admin/orders/[category]

5. Admin génère/obtient les données (credentials ou clé)

6. Admin saisit dans un formulaire modal:
   - Pour Rental: username + password
   - Pour License: activationKey

7. Admin soumet et statut devient "completed"

8. Email notification envoyée avec les données

9. Utilisateur voit les données affichées dans la commande

10. Utilisateur peut consulter ses identifiants/clé dans /account/orders/[category]

═══

SÉCURITÉ & AUTHENTIFICATION

- Authentification obligatoire: Utilisateur DOIT être connecté pour commander
- Stratégie: JWT (JSON Web Tokens) avec refresh tokens

- Isolation des données: Utilisateur ne voit que ses propres commandes
- Vérification userId: Chaque requête vérifie l'authentification
- Autorisation: Admin-only pour /admin/* routes
- Validation côté backend: Tous les inputs sont validés
- HTTPS obligatoire en production
- Rate limiting sur les endpoints sensibles (création commande, login)

═══════════════════════════════════════

═══════════════════════════════════════

═══════════════════════════════════════

═══════

STRUCTURE DES DONNÉES (MONGODB)

SERVICE:

```
{
  _id: ObjectId,
  name: String,
  description: String,
  price: Number,
  category: Enum ["IMEI", "Server", "Rental", "License"],
  active: Boolean,
  createdAt: Date,
  updatedAt: Date
}
```

ORDER:

```
{
  _id: ObjectId,
  userId: ObjectId (référence User),
  serviceId: ObjectId (référence Service),
  serviceCategory: Enum ["IMEI", "Server", "Rental", "License"],
  status: Enum ["pending", "processing", "completed"],

  userSubmittedData: {
    // IMEI: { imei: "123456789012345", imageUrl: "https://imgbb.com/..." }
    // Server: { username: "user", email: "user@example.com" }
```

```
    // Rental/License: {} (vide)
  },

  deliveryData: {
    // Rental: { username: "...",
password: "..." }
    // License: { activationKey: "..." }
    // IMEI/Server: null
  },

  createdAt: Date,
  updatedAt: Date
}

USER:
{
   _id: ObjectId,
```

```
  email: String (unique),
  password: String (hashed),
  balance: Number (solde en €),
  role: Enum ["user", "admin"],
  createdAt: Date,
  updatedAt: Date
}
```

═══════════════════════════════════════

═══════════════════════════════════════

═══════════════════════════════════════

═══════════

ROUTES API REQUISES

AUTHENTICATION:

POST /api/auth/register - Créer compte utilisateur
POST /api/auth/login - Connexion
POST /api/auth/logout - Déconnexion
POST /api/auth/refresh - Refresh token
GET /api/auth/me - Profil utilisateur actuel

SERVICES:
GET /api/services - Tous les services
GET /api/services?category=IMEI - Services filtrés par catégorie
POST /api/admin/services - Créer service (admin only)
PUT /api/admin/services/:id -

Modifier service (admin only)
  DELETE /api/admin/services/:id - Désactiver service (admin only)

ORDERS (USER):
  POST /api/orders - Créer une commande
  GET /api/orders/my - Mes commandes
  GET /api/orders/my?category=IMEI - Mes commandes filtrées
  GET /api/orders/:id - Détail d'une commande

ORDERS (ADMIN):
  GET /api/admin/orders - Toutes les commandes

GET /api/admin/orders?category=IMEI - Filtrées par catégorie

GET /api/admin/orders?status=pending - Filtrées par statut

GET /api/admin/orders/:id - Détail commande

PUT /api/admin/orders/:id/status - Changer statut

POST /api/admin/orders/:id/deliver - Saisir données livraison (Rental/License)

═══════════════════════════════════

═══════════════════════════════════

═══════════════════════════════════

═══════════

PAGES/ROUTES FRONTEND REQUISES

PUBLIC:
  / - Accueil
  /login - Connexion
  /register - Inscription

UTILISATEUR (authenticated):
  /services/imei - Affiche services IMEI
  /services/server - Affiche services Server
  /services/rental - Affiche services Rental

/services/license - Affiche services License

/services/:id/order - Page commande (formulaire dynamique)

/account/orders - Historique commandes (toutes catégories)

/account/orders/imei - Historique IMEI

/account/orders/server - Historique Server

/account/orders/rental - Historique Rental

/account/orders/license - Historique License

/account/orders/:id - Détail commande

/account/profile - Profil utilisateur (solde, infos)

ADMINISTRATEUR (admin only):
/admin/services - Gestion des services
/admin/services/new - Créer service
/admin/services/:id/edit - Modifier service

/admin/orders - Historique commandes (toutes catégories)
/admin/orders/imei - Commandes IMEI à traiter
/admin/orders/server - Commandes Server à traiter

/admin/orders/rental - Commandes Rental à traiter
/admin/orders/license - Commandes License à traiter
/admin/orders/:id - Traiter une commande (modal/formulaire)

═══

PRINCIPES TECHNIQUES CLÉS
- Backend unifié: Une seule logique Services et Orders
- Filtrage par catégorie: Jamais par

duplication de modèles
- Formulaires dynamiques: Jamais de code statique
- Architecture extensible et maintenable
- Validation côté frontend ET backend
- Gestion complète des erreurs
- UX claire inspirée des plateformes professionnelles IMEI
- Code réutilisable et modulaire

═══════════════════════════════════════
═══════════════════════════════════════
═══════════════════════════════════════
═══════

OBJECTIF FINAL

Obtenir une plateforme professionnelle, scalable et propre, respectant les bonnes pratiques MERN Stack,
capable de gérer plusieurs types de services avec une seule architecture cohérente.